

Relatório 11 - Prática: Lidando com Dados do Mundo Real (II)

Igor Carvalho Marchi

1. Introdução

Aulas de um minicurso com conceitos importantes sobre como entender e manipular dados do mundo real ao lidar com projetos de Machine Learning e Inteligência Artificial. O curso enfatiza que o sucesso de um projeto depende da qualidade dos dados utilizados. Por isso, são discutidos desafios comuns encontrados em bases reais, como dados desbalanceados, valores ausentes, outliers e problemas de escala, além das principais estratégias para tratá-los de forma adequada.

2. Desenvolvimento

2.1 More Data Mining and Machine Learning Techniques

Aula 1 – K-Nearest Neighbors (KNN) Concepts

Nesta aula é apresentado o algoritmo K-Nearest Neighbors (KNN), que significa “K vizinhos mais próximos”. O algoritmo é utilizado para classificação ou regressão, comparando um novo dado com dados já conhecidos.

O funcionamento ocorre da seguinte forma:

1. Define-se um valor K, que representa quantos vizinhos serão analisados.
2. Calcula-se a distância entre o novo ponto e os pontos já existentes no conjunto de dados.
3. O algoritmo escolhe os K pontos mais próximos.
4. A classificação do novo ponto é feita com base na maioria das classes desses vizinhos.

Aula 2 – Activity Using KNN to predict a rating for a movie

Nesta atividade é mostrado um exemplo prático de uso do KNN para prever a avaliação de um filme. Primeiramente é criado um DataFrame com informações sobre filmes, incluindo o ID do filme e sua avaliação média.

Depois disso, os dados são organizados de forma que cada filme tenha características que permitam medir sua similaridade com outros filmes.

Com base nessas informações, é criada uma função que calcula a distância entre filmes e retorna os K filmes mais parecidos com o filme escolhido.

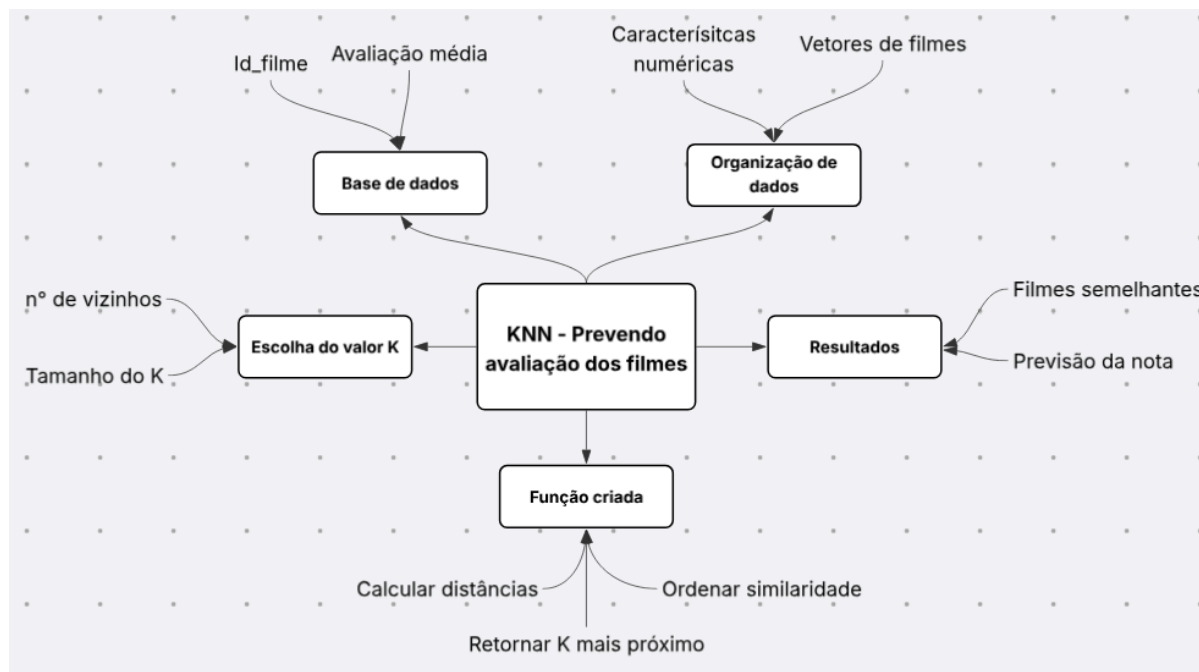


Imagem 1 - Feito pelo próprio autor.

Assim, é possível prever a avaliação de um filme ou recomendar filmes semelhantes com base nos vizinhos mais próximos.

Aula 3 – Dimensionality Reduction: Principal Component Analysis (PCA)

Nesta aula é apresentado o conceito de redução de dimensionalidade.

Muitos conjuntos de dados possuem um grande número de características (features), o que pode dificultar a análise e o treinamento de modelos. Esse problema é conhecido como Curse of Dimensionality (Maldição da Dimensionalidade).

Para resolver esse problema, pode-se utilizar o PCA (Principal Component Analysis).

O PCA é uma técnica que transforma um conjunto de variáveis possivelmente correlacionadas em um novo conjunto de variáveis chamadas componentes principais, que não possuem correlação entre si.

Essa técnica permite reduzir o número de dimensões mantendo a maior parte da informação dos dados, facilitando a visualização e o treinamento de modelos.

Aula 4 – Activity PCA Example with the Iris dataset

Nesta atividade é aplicado o PCA utilizando o famoso dataset Iris.

O dataset Iris possui 4 características das flores:

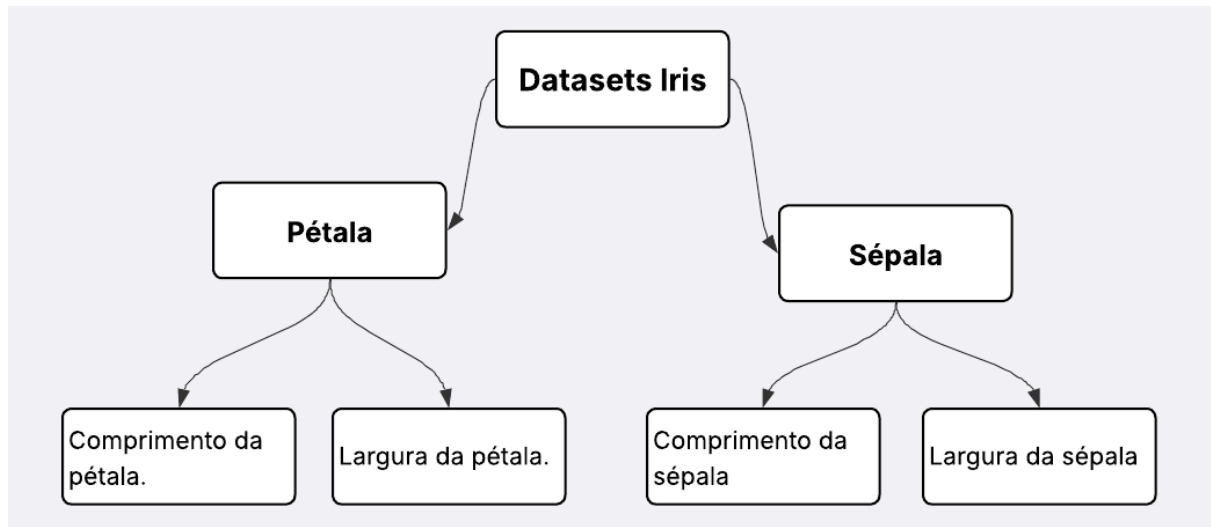


Imagem 2 - Criado pelo próprio autor.

Com o PCA, essas 4 dimensões são reduzidas para 2 dimensões, mantendo a maior parte da variação dos dados. Depois disso é gerado um gráfico de dispersão (scatter plot) que permite visualizar as diferentes espécies de flores em duas dimensões, facilitando a análise dos dados.

Aula 5 – Data Warehousing Overview: ETL and ELT

Nesta aula é explicado o conceito de Data Warehouse, que é um grande repositório de dados utilizado para armazenar informações provenientes de várias fontes diferentes.

Dois processos importantes nesse contexto são ETL e ELT.

ETL – Extract, Transform, Load

No processo ETL os dados passam pelas seguintes etapas:

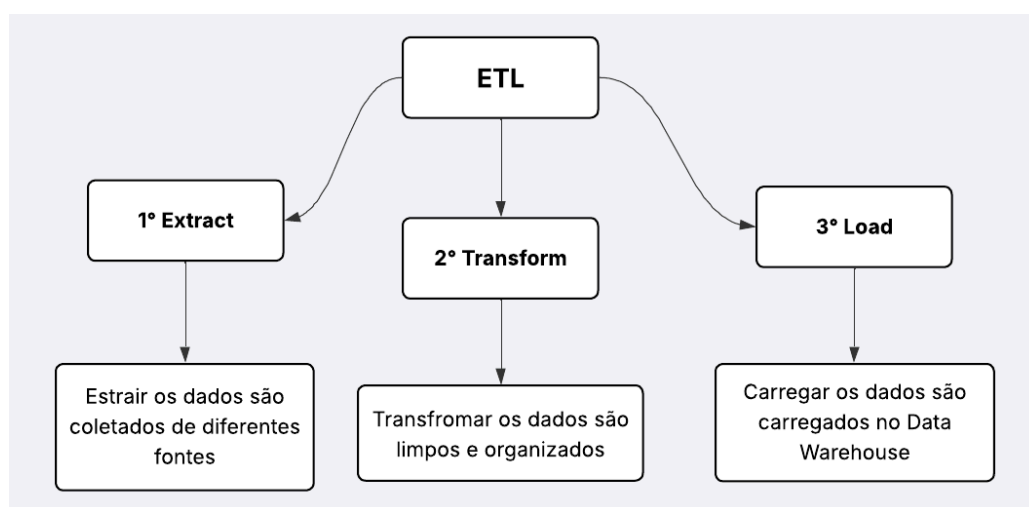


Imagem 3 - Feito pelo próprio autor.

ELT – Extract, Load, Transform

No ELT o processo ocorre de forma diferente:

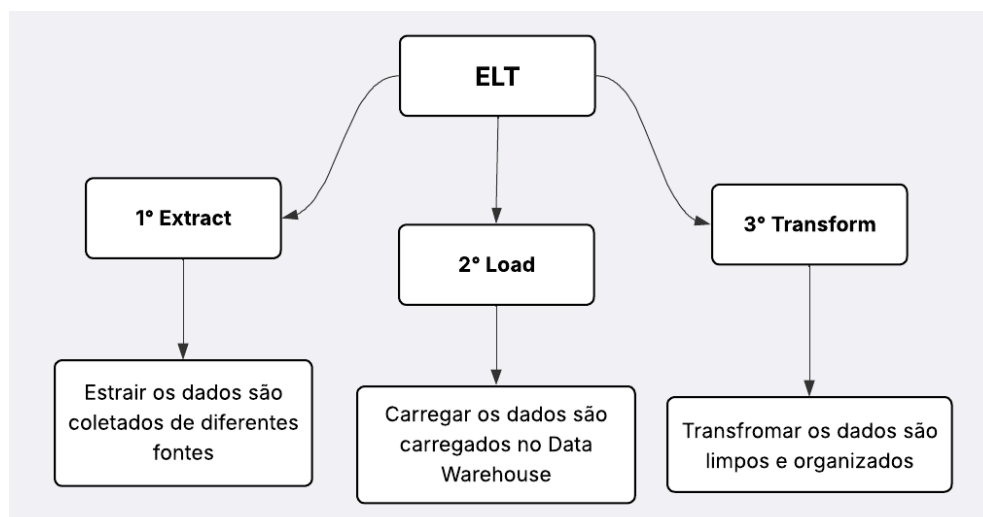


Imagem 4 - Feito pelo próprio autor.

Aula 6 - Reinforcement Learning

‘Reinforcement Learning’ na tradução literal é o aprendizado por reforço, uma das técnicas de machine learning, ele imita as ações humanas de tentativa e erro, funciona da seguinte forma, o agente de aprendizado interage com o ambiente e toma ações aprendendo com a experiência, ele é penalizado quando erra e recompensado quando acerta, com um certo tempo o agente aprende a tomar as decisões corretas para a situação, o exemplo do vídeo é o jogo do Pacman.

Ele também fala sobre o Q-Learning, um algoritmo sem modelo baseado em valores de Q, onde Q significa qualidade, onde o agente aprende a tomar decisões baseadas em qual estado de Q tem o maior valor. Um dos problemas do aprendizado por reforço pode ser o ‘Exploration Problem’ que é a preocupação com a eficiência de explorar todos os estados possíveis, a melhor maneira de resolver é utilizando um termo epsilon que consiste em se um número aleatório for menor que epsilon não siga o maior ‘Q’, apenas faça um caminho aleatório, assim dessa forma a exploração nunca acaba.

Aula 7 – Activity Reinforcement Learning & Q-Learning with Gym

Nos é apresentado a biblioteca do python ‘Gym’ onde é possível simular ambientes de aprendizado por reforço

O ambiente utilizado é o **Taxi-v3**, onde um táxi precisa:

1. Encontrar um passageiro que pode estar nas posições R/G/Y/B
2. Pegá-lo na posição que estiver na cor rosa.
3. Levá-lo até o destino correto, que no caso é a cor azul.

```
# biblioteca que tem o joguinho do taxi
import gym
import random

random.seed(1234)

# criando o ambiente do taxi
streets = gym.make("Taxi-v3", render_mode='ansi').env

# resetando o ambiente
streets.reset()

# mostra o estado atual do jogo
print('\n' + streets.render())
```

```
+-----+
|R: | :G|
| : | : |
| : : : |
| | : : |
|Y| : |B: |
+-----+
```

Dentro dessa aula nos é mostrado como que é feito para simular um aprendizado por reforço, e após é mostrado o resultado, do táxi indo e pegando o passageiro e deixando ele no lugar esperado, sempre evitando a menor quantidade de movimentos e desviando dos obstáculos.

Aula 8 – Understanding a Confusion Matrix

A Matriz de Confusão é uma ferramenta utilizada para avaliar o desempenho de modelos de classificação. Ela compara os valores previstos pelo modelo com os valores reais.

A matriz possui quatro elementos principais:

- **True Positive (TP)** – quando o modelo acerta um positivo
- **True Negative (TN)** – quando o modelo acerta um negativo
- **False Positive (FP)** – quando o modelo prevê positivo mas é negativo
- **False Negative (FN)** – quando o modelo prevê negativo mas é positivo

	Actual YES	Actual NO
Predicted YES	TRUE POSITIVES	FALSE POSITIVES
Predicted NO	FALSE NEGATIVES	TRUE NEGATIVE

Matriz de confusão é uma tabela usada para mostrar as previsões de algo com os resultados reais, pode ser usada tanto para um teste de doença ou para a visão computacional onde ele tem que dizer se tem algo ali ou não.

Aula 9 – Measuring Classifiers (Precision, Recall, F1, ROC, AUC)

Para avaliar modelos de classificação, existem diversas métricas.

Precision

A Precision mede quantos dos resultados positivos previstos estão corretos.

$$Precision = \frac{TP}{TP + FP}$$

Recall

O Recall mede quantos dos positivos reais foram corretamente identificados.

$$Recall = \frac{TP}{TP + FN}$$

F1 Score

O F1 Score é a média harmônica entre Precision e Recall.

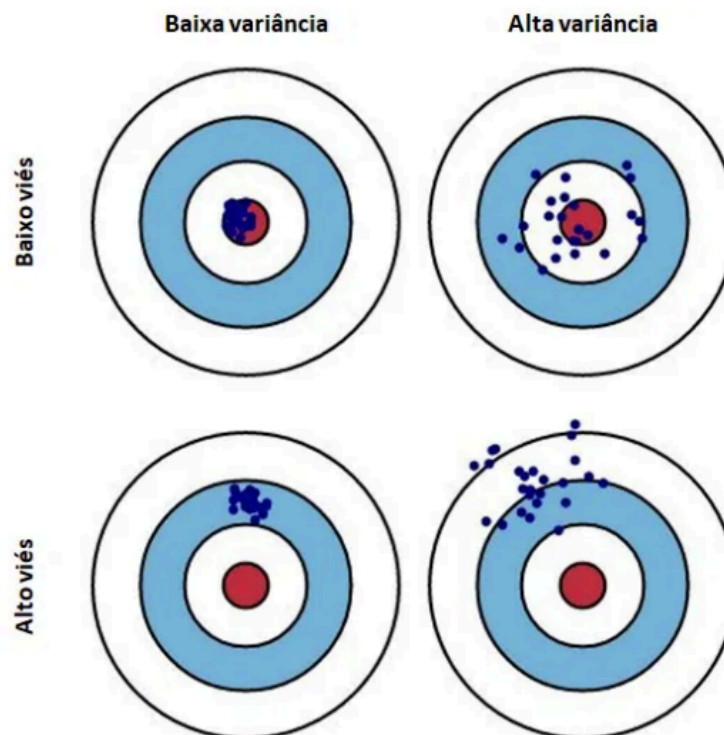
$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

Essa métrica é útil quando queremos equilibrar Precision e Recall.

2.2 Dealing with Real-World Data

Aula 1 – Bias Variance Tradeoff

Bias Variance ou viés e variância, viés é o quão longe a média da sua previsão está longe dos resultados reais, e a variância quão dispersos estão seus valores previstos da resposta real, exemplo:



Bias (Viés) representa o erro causado por suposições simplificadas do modelo.

Variance (Variância) representa o quanto o modelo varia dependendo do conjunto de dados utilizado.

Aula 2 – K-Fold Cross-Validation to avoid overfitting

É apresentado um dos problemas do machine learning, o overfitting, é um problema que ocorre quando um modelo é treinado muito de perto para um conjunto de dados, fazendo com que ele tenha um desempenho ruim em novos dados. Uma das maneiras de evitar esse problema é o K-fold cross-validation é uma técnica de validação onde os dados são divididos em K partes iguais.

Funcionamento:

1. O conjunto de dados é dividido em K partes iguais.
2. Em cada iteração:
 - 1 parte é usada para teste.
 - K-1 partes são usadas para treino.
3. O processo se repete K vezes.
4. No final, calcula-se a média das métricas de avaliação.

INTERAÇÃO 1	TEST	TRAIN	TRAIN	TRAIN	TRAIN
INTERAÇÃO 2	TRAIN	TEST	TRAIN	TRAIN	TRAIN
INTERAÇÃO 3	TRAIN	TRAIN	TEST	TRAIN	TRAIN
INTERAÇÃO 4	TRAIN	TRAIN	TRAIN	TEST	TRAIN
INTERAÇÃO 5	TRAIN	TRAIN	TRAIN	TRAIN	TEST

Imagem 5 - Feito pelo próprio autor(Referência do vídeo).

Aula 3 – Data Cleaning and Normalization

A aula inicia destacando que a maior parte do trabalho de um cientista de dados não está na criação do modelo, mas sim na preparação dos dados. Muitas vezes, a qualidade do modelo depende diretamente da qualidade dos dados de entrada.

Em situações reais, os dados raramente estão prontos para uso. Eles podem apresentar diversos problemas que comprometem a análise e a capacidade de generalização do modelo.

Principais problemas encontrados nos dados:

- **Dados discrepantes (outliers):** valores muito diferentes do padrão esperado.
- **Dados faltantes (missing data):** ausência de informações em determinadas colunas.
- **Dados maliciosos ou corrompidos:** registros alterados ou inválidos.
- **Dados irrelevantes:** informações que não contribuem para o problema analisado.
- **Dados inconsistentes:** divergências de formato, unidade ou padrão (ex: datas em formatos diferentes).
- **Dados duplicados:** registros repetidos que podem distorcer análises estatísticas.

Aula 4 – Activity Cleaning Web Log Data

Nessa aula vai mostrar a limpeza dos dados, é trazido um arquivo sobre os acesso de um site dele, e é mostrado alguns scripts filtrando e limpando os dados.

```
URLCounters = {}
with open(logPath, 'r') as f:
    for line in (l.rstrip() for l in f):
        match = format_pat.match(line)
        if match:
            access = match.groupdict()
            request = access['request']
            fields = request.split()
            if (len(fields) == 3): # verifica se existem 3 campos
                URL = fields[1] # url é o segundo campo da requisicao
                if URL in URLCounters:
                    URLCounters[URL] += 1
                else:
                    URLCounters[URL] = 1
results = sorted(URLCounters, key=lambda i :int(URLCounters[i]), reverse=True)
for result in results[:20]:
    print(result + " : " + str(URLCounters[result]))
```

Ao decorrer da aula vamos aprendendo a filtrar os dados tirando os bots e deixando apenas os humanos, chegando no código abaixo, onde está demonstrando qual das notícias do site é a mais acessada.

```
URLCounts = {}

with open(logPath, "r") as f:
    for line in (l.rstrip() for l in f):
        match = format_pat.match(line)
        if match:
            access = match.groupdict()
            agent = access['user_agent']
            if (not ('bot' in agent or 'spider' in agent or
                    'Bot' in agent or 'W3 Total Cache' in agent or agent == '-')):
                request = access['request']
                fields = request.split()
                if (len(fields) == 3):
                    (action, URL, protocol) = fields

                    # adicionando a verificação para ver se é um site (termina com "/")
                    if (URL.endswith("/")):
                        if (action == 'GET'):
                            if URL in URLCounts:
                                URLCounts[URL] = URLCounts[URL] + 1
                            else:
                                URLCounts[URL] = 1

results = sorted(URLCounts, key=lambda i: int(URLCounts[i]), reverse=True)

for result in results[:20]:
    print (result + " : " + str(URLCounts[result]))
```

Assim demonstrando os sites mais acessados por pessoas:

```
/orlando-headlines/: 36
/comics-2/: 12
/world/: 12
/weather/: 4
/australia/: 4
/about/: 4
/national-headlines/: 3
/feed/: 2
/sample-page/feed/: 2
/science/: 2
/technology/: 2
/entertainment/: 1
/san-jose-headlines/: 1
/business/: 1
/travel/feed/: 1
```

Aula 5 – Normalizing Numerical Data

Nesta aula é reforçada a importância da normalização. Quando um modelo utiliza múltiplos atributos numéricos com escalas muito diferentes, isso pode influenciar negativamente o cálculo de distâncias ou pesos. O exemplo apresentado é o de idade e renda:

- Idade varia entre 0 e 100
- Renda pode variar de 0 até bilhões

Sem normalização, a renda pode dominar os cálculos do modelo. A aula também destaca que nem todos os algoritmos precisam de normalização, por isso é sempre importante consultar a documentação do método utilizado.

Aula 6 – Detecting Outliers

Nesta aula é apresentado um exemplo prático de remoção de valores discrepantes (outliers) utilizando o desvio padrão. Primeiramente, o código calcula a mediana do conjunto de dados e o desvio padrão. Em seguida podemos ver o código da função abaixo:

```
def reject_outliers(data):  
    u = np.median(data) # calcula a mediana  
    s = np.std(data) # calcula o desvio padrão  
    # verifica se esta abaixo ou acima dos desvios padrões da mediana e depois retorna um conjunto de dados filtrados  
    filtered = [e for e in data if (u - 2 * s < e < u + 2 * s)]  
    return filtered  
  
filtered = reject_outliers(incomes)  
plt.hist(filtered, 50)  
plt.show()
```

Valores que ficam fora desse intervalo são considerados discrepantes e removidos do conjunto. Após a filtragem, é gerado um histograma para visualizar a distribuição dos dados sem os valores extremos.

Aula 7 – Feature Engineering and the Curse of Dimensionality

A engenharia de atributos consiste em aplicar conhecimento sobre os dados para criar variáveis mais adequadas ao modelo. Isso inclui decidir quais atributos usar, como tratar dados faltantes e até combinar variáveis para melhorar o desempenho.

Essa etapa é fundamental, pois não basta utilizar todos os dados disponíveis; é necessário selecionar e transformar os atributos de forma estratégica. Também é importante para evitar a maldição da dimensionalidade, que ocorre quando o número de atributos é muito grande. Cada novo atributo representa uma nova dimensão, e muitas dimensões tornam os dados mais dispersos e difíceis de analisar.

Aula 8 – Imputation Techniques for Missing Data

Como visto em outras aulas quando lidamos com dados faltantes, podemos usar uma solução simples que é substituir os valores ausentes pela média ou mediana da coluna. É um método rápido, mas pouco preciso. Outra opção é remover os registros com valores faltantes, embora isso possa causar perda de informação.

Também é possível utilizar o KNN para estimar valores com base nos vizinhos mais próximos, método adequado para dados numéricos. Para dados categóricos, podem ser usados modelos de Machine Learning mais avançados.

Aula 9 – Handling Unbalanced Data: Oversampling, Undersampling e SMOTE

Nessa aula é abordado o problema de dados desbalanceados, que ocorre quando há grande diferença entre a quantidade de exemplos de cada classe. Um exemplo é a detecção de fraudes, onde a maioria das transações é legítima e apenas uma pequena parte representa fraude. Quando o modelo é treinado com esse tipo de base, ele pode aprender a prever quase sempre a classe majoritária, comprometendo sua capacidade de identificar corretamente os casos minoritários.

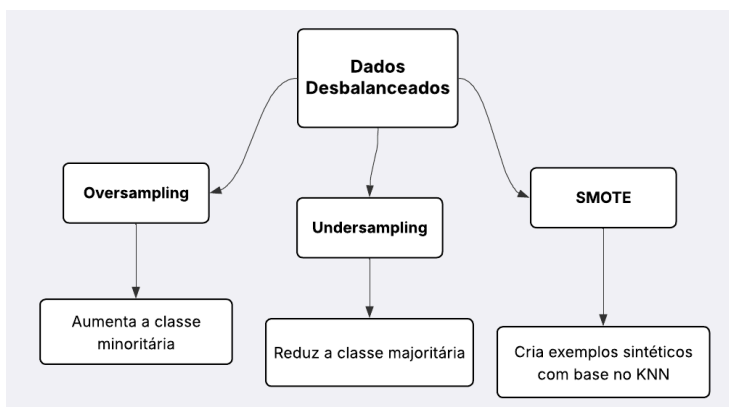


Imagem 6 - Feito pelo próprio autor.

Para lidar com esse problema, são apresentadas três principais técnicas: o Oversampling, que aumenta artificialmente a quantidade de exemplos da classe minoritária; o Undersampling, que reduz a quantidade de exemplos da classe majoritária; e o SMOTE, que cria novos exemplos sintéticos da classe minoritária com base nos vizinhos mais próximos, tornando o conjunto de dados mais equilibrado sem apenas duplicar registros.

Aula 10 – Binning, Transforming, Encoding, Scaling and Shuffling

são apresentadas técnicas complementares utilizadas no pré-processamento dos dados, etapa fundamental da engenharia de atributos. Essas técnicas ajudam a preparar melhor o conjunto de dados antes do treinamento do modelo, garantindo que as informações estejam no formato adequado e contribuam de maneira equilibrada para o aprendizado.

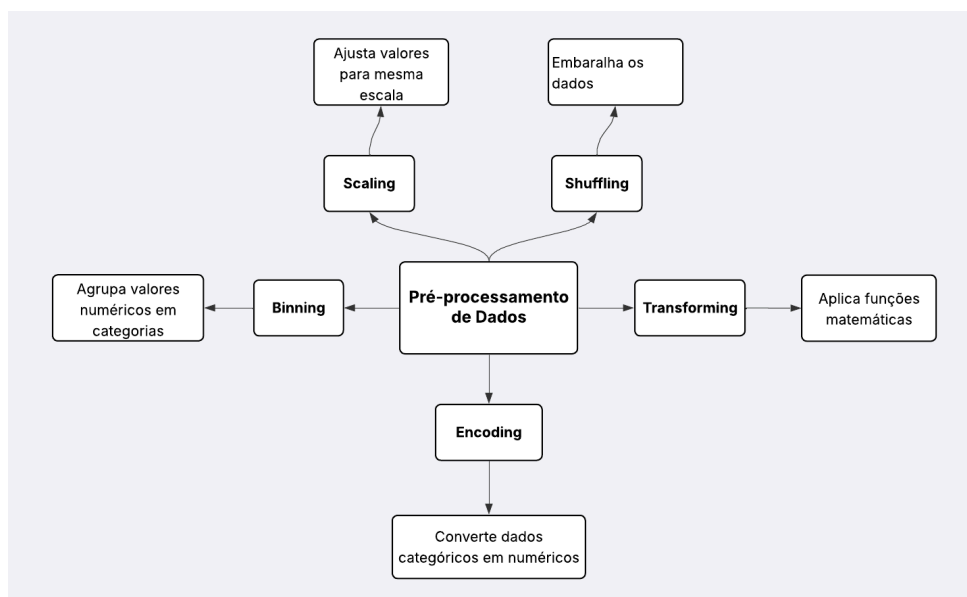


Imagem 7 - Feito pelo próprio autor.

Entre as técnicas abordadas está o Binning, que consiste em transformar valores numéricos em categorias, como agrupar idades por faixas etárias. Também é apresentado o Transforming, que envolve aplicar funções matemáticas aos dados para melhorar sua distribuição. O Encoding é utilizado para converter variáveis categóricas em valores numéricos, tornando-as compatíveis com os algoritmos de Machine Learning.

Além disso, a aula explica o Scaling, que ajusta os valores numéricos para uma mesma escala, evitando que variáveis com números maiores tenham mais influência no modelo. Por fim, o Shuffling reorganiza aleatoriamente os dados antes do treinamento, impedindo que a ordem original interfira no processo de aprendizado.

3. Conclusão

Ao longo das aulas foi possível compreender que os projetos em Machine Learning e Inteligência Artificial não dependem apenas da escolha do algoritmo, mas principalmente da qualidade e do tratamento adequado dos dados. Técnicas como KNN, PCA, Reinforcement Learning e métricas de avaliação demonstram como os modelos funcionam na prática, mas também evidenciam que sua eficiência está diretamente ligada à preparação correta das informações.

A segunda parte do conteúdo reforça ainda mais essa ideia ao abordar problemas comuns encontrados em bases reais, como dados desbalanceados, valores ausentes, outliers e diferenças de escala. Foram apresentadas diversas estratégias para lidar com essas situações, como normalização, validação cruzada, engenharia de atributos, SMOTE e outras técnicas de pré-processamento.

Referências

Vídeo da seção 6 - More Data Mining and Machine Learning Techniques .

Vide da seção 7 - Dealing with Real-World Data.